

Communication and Information Engineering

Embedded Systems
Course Code: CIE 349

VMO

Autonomous Home Pet and Safety Monitoring Robot Technical Documentation

Project Team:

Matthew Nader	202300103
Omar Tamer Samir	202300713
Mohamed Gadwal	202301231
Amr Emad Mohamed	202301016
Mohamed Magdy	202101520

Contents

1	Introduction	1
2	Hardware Architecture & Bill of Materials	1
2.1	Distributed System Processing Tiers	1
2.2	Bill of Materials (BOM)	1
3	PCB Design, Schematics, and Signal Integrity	2
3.1	Power Distribution and Safety Interlocks	2
3.2	Discrete MOSFET Logic Level Shifting	2
4	Embedded Firmware Architecture	3
4.1	PIC16F877A Hard Real-Time Layer	3
4.2	ESP32-S3 Nervous System and Edge Middleware	3
5	Cognitive Perception and ROS 2 Autonomy	4
5.1	Containerized Runtime Environment	4
5.2	Edge-AI: YOLO26 Nano and Unique Frames Processing	4
5.3	ROS 2 Subsumption Control and Kinematics Engine	4
6	Mobile Core Intelligence and Frontend	5
6.1	Speech Processing and LLM Agent	5
6.2	Locked 60 Hz Vector Rendering	5
7	Validation and Conclusion	5
7.1	Iterative Improvements and Stabilization	5
7.2	Conclusion	7
7.3	Future Work	7
A	Appendix: Master System Interconnect Diagram	8

1 Introduction

VMO is an interactive, autonomous home pet robot engineered to engage dynamically with users using computer vision and a localized sensor array. Drawing stylistic inspiration from classic compact robotic companions, VMO acts simultaneously as an engaging household companion and a safety monitor. By continually logging environmental factors, the system alerts users to anomalies, bridging consumer robotics with proactive smart-home defense.

The platform is built around a heterogeneous, multi-tier computing hierarchy. Hard real-time sensor polling and motor actuation are governed by an 8-bit PIC microcontroller; asynchronous task distribution and wireless communication are handled by a dual-core ESP32-S3 running FreeRTOS; and high-level cognitive perception—including real-time computer vision, ROS 2 navigation, and cloud telemetry—runs inside a containerized Linux environment on a Raspberry Pi 4. A consumer Android smartphone mounted to the chassis serves as VMO's expressive face, voice interface, and local network hotspot.

*This document is a focused architectural report prepared to meet submission constraints. For the complete repository of exhaustive trace schematics, full mathematical models, component netlists, and uncut code layers, please refer to the **Full Extended Technical Documentation**.*

2 Hardware Architecture & Bill of Materials

2.1 Distributed System Processing Tiers

The VMO system relies on a heterogeneous processing topology that decouples high-latency, computation-heavy algorithms from time-critical, hardware-level execution loops.

- **Muscle Layer (Hard Real-Time):** Governed by an 8-bit Microchip PIC16F877A microcontroller, dedicated strictly to physical sensor polling and deterministic motor drive execution.
- **Nervous System (Task Distribution):** Governed by a dual-core Espressif ESP32-S3 running FreeRTOS, providing hardware isolation, asynchronous queue buffering, and cross-platform communication serialization.
- **Brain Layer (High-Level Cognitive Perception):** Governed by a Raspberry Pi 4 executing a headless Linux containerized environment. Handles autonomous path evaluation and distributed ROS 2 microservice pipelines.
- **Face UI Layer (Passive Expressions):** Governed by an Android smartphone mounted directly to the chassis, processing high-level emotional feedback without consuming power from the core embedded boards.

2.2 Bill of Materials (BOM)

The architectural hardware components driving the processing and peripheral pipelines include:

- **Processing Cores:** Microchip PIC16F877A (5V logic), ESP32-S3-N8R2 (3.3V module), Raspberry Pi 4 (2 GB RAM).
- **Power Management:** 11.1V 3S LiPo battery with BMS, 10A inline fast-blow fuse, 15SQ045 high-current Schottky diode, dual independent 5V and 3.3V buck regulators, and INA226 digital power monitor (I^2C).
- **Sensors:** MQ-2 hazardous gas sensor (analog), DHT11 digital temperature/humidity sensor, 4-channel HC-SR04 ultrasonic array, dual high-speed optical wheel encoders.
- **Actuation:** L298N H-Bridge dual motor driver powering differential drive chassis locomotion.

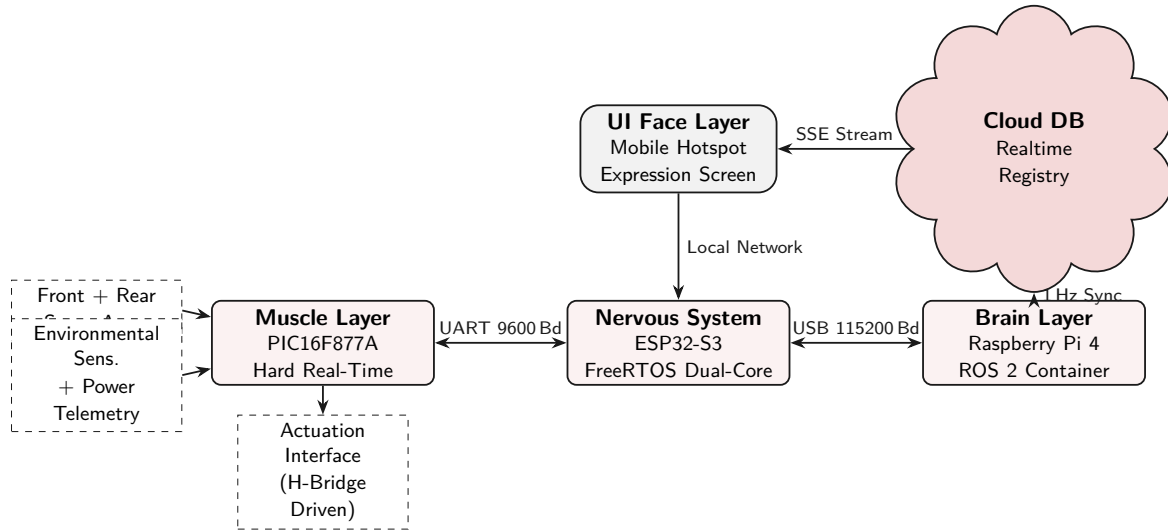


Figure 1: Distributed Heterogeneous Computing Hierarchy and Multi-Protocol Communication Network.

3 PCB Design, Schematics, and Signal Integrity

3.1 Power Distribution and Safety Interlocks

Primary power from the 11.1V 3S LiPo travels through a deeply layered safety hierarchy (Figure 2). A mechanical kill switch breaks the main feed, followed by a 10A fast-blow fuse to prevent thermal runaway. Post-protection, power splits. Raw 11.1V routes directly to the L298N motor drivers, explicitly keeping high-current inductive surges off the logic planes. The remaining power feeds parallel 5V and 3.3V buck regulators for the microcontrollers.

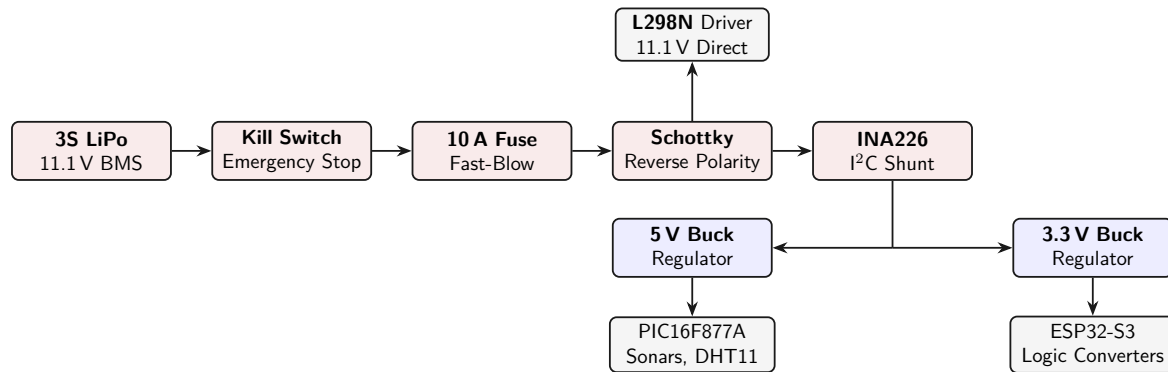


Figure 2: Power Distribution Chain and Dual-Rail Regulation Architecture.

3.2 Discrete MOSFET Logic Level Shifting

Interfacing the 5V PIC logic domain with the 3.3V ESP32 domain required discrete bidirectional level shifting to prevent electrical overstress. The custom PCB utilizes 2N7002 N-channel MOSFETs with 10 k Ω pull-up networks. The mathematical operation functions across distinct states:

1. **Low-Side Drive (3.3V Driving Low):** The ESP32 pulls the 3.3V node to 0V.

$$V_{gs} = V_{Gate} - V_{Source} = 3.3V - 0V = 3.3V > V_{gs(th)} \approx 2.1V$$

The channel saturates, conducting through the open drain and pulling the 5V PIC node to 0V.

2. **High-Side Drive (5V Driving Low):** The PIC pulls the 5V node to 0V. The MOSFET's internal parasitic body diode becomes forward-biased, clamping the 3.3V node to $V_{Low} \approx 0.7V$. This drops the source below gate potential, triggering $V_{gs} \approx 2.6V > V_{gs(th)}$, fully opening the channel to drive the 3.3V node to a clean 0V.

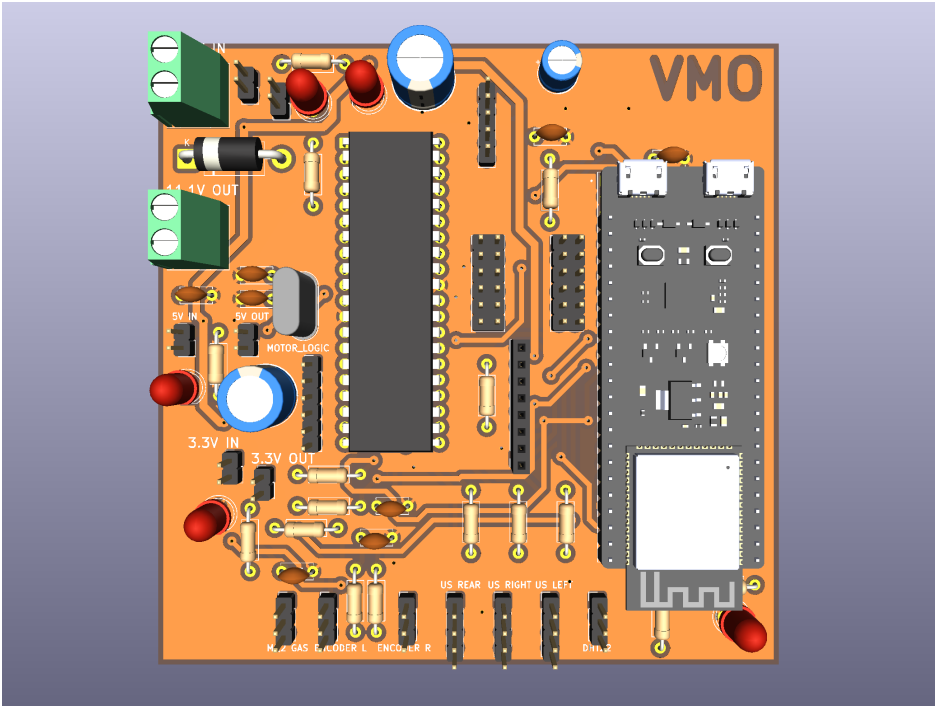


Figure 3: Complete 3D Render of the Populated VMO Baseboard.

The custom two-layer PCB (Figure 3) employs a "Star Power" grounding topology, physically separating high-current motor return paths from digital logic grounds to eliminate ground bounce.

4 Embedded Firmware Architecture

4.1 PIC16F877A Hard Real-Time Layer

The PIC firmware implements a strict split-responsibility paradigm. Due to its limited 8-level hardware call stack, context switching via a preemptive RTOS is impossible. Instead, firmware is divided into four structural layers (Figure 4), governed by a lightweight cooperative time-based task dispatcher.

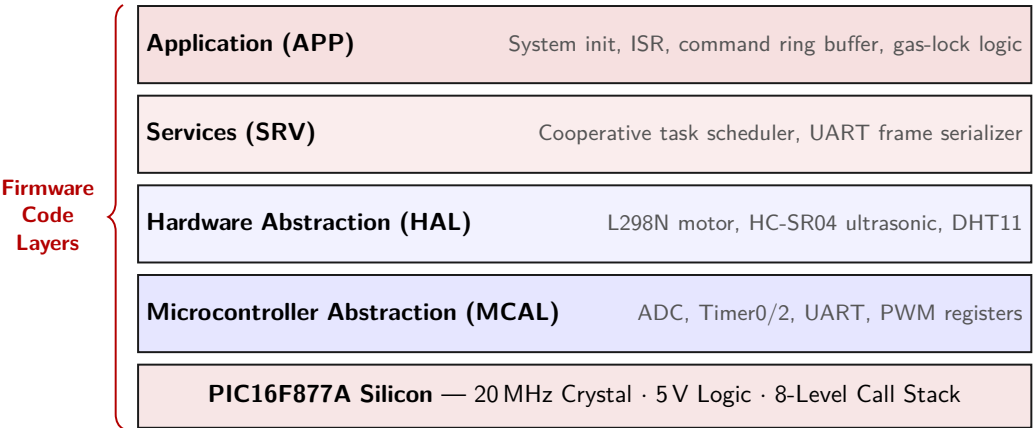


Figure 4: PIC16F877A Layered Software Architecture.

Sensor polling is scheduled dynamically: the MQ-2 gas ADC runs every 100 ms, ultrasonics every 200 ms, and DHT11 every 2000 ms. Serialized telemetry strings deliberately avoid 'sprintf' to prevent stack overflows, utilizing custom integer modulo-division loops instead. Commands from the ESP32 are captured via an interrupt-driven UART ring buffer, entirely bypassing main-loop blocking delays.

4.2 ESP32-S3 Nervous System and Edge Middleware

The dual-core ESP32-S3 manages asynchronous task buffering via a deterministic FreeRTOS scheduler:

- **Core 0:** Dedicated to physical interplay. It runs 'vTaskMotorCmd' (Priority 4) for zero-latency translation of drive codes to the PIC, and 'vTaskSensorReader' (Priority 3).
- **Core 1:** Dedicated to network routing. It runs 'vTaskPiComm' (Priority 2) to serialize sensor data into Protobuf+COBS binary frames over USB to the Raspberry Pi.

To minimize ultrasonic cross-talk, the ESP32 enforces strict 15 ms phase delays between triggering the Left, Right, and Rear sonar channels, allowing stray acoustic energy to fully dissipate.

5 Cognitive Perception and ROS 2 Autonomy

5.1 Containerized Runtime Environment

The cognitive framework of VMO runs inside a headless Linux Docker container on the Raspberry Pi 4. This sandbox isolates complex Python and C++ dependencies from the host OS, ensuring identical compilation optimization and eliminating dependency conflicts.

Dockerfile — Production Environment Blueprint

```
FROM ros:humble-ros-base-jammy
ENV DEBIAN_FRONTEND=noninteractive

# 1. System packages, build tools, and vision libraries
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential cmake git curl nano python3-pip \
    python3-colcon-common-extensions v4l-utils \
    libopencv-dev protobuf-compiler \
    && apt-get clean && rm -rf /var/lib/apt/lists/*

# 2. Python robotics and cloud dependencies
RUN pip3 install --no-cache-dir \
    pyserial cobs protobuf firebase-admin numpy opencv-python-headless

# 3. Project workspace and YOLO-CPP Vision Core integration
WORKDIR /root/robot_ws
RUN git clone https://github.com/Geekineer/YOLOs-CPP.git src/YOLOs-CPP \
    && mkdir -p src/pet_brain/protos

# 4. ROS 2 environment sourcing
RUN echo "source /opt/ros/humble/setup.bash" >> /root/.bashrc

# 5. Communication ports
EXPOSE 5000/udp 5001/udp

ENTRYPOINT ["/ros_entrypoint.sh"]
CMD ["bash"]
```

Table 1: Containerized ROS 2 Stack Deployment Blueprint.

Hardware passthrough flags (`-device=/dev/video0` and `-device=/dev/ttyUSB0`) provide the container with direct, policed access to the webcam and ESP32 USB link without exposing the entire host peripheral bus.

5.2 Edge-AI: YOLO26 Nano and Unique Frames Processing

The vision tracking system is written entirely in native C++ using the ONNX Runtime API to prevent out-of-memory kernel panics on the 2GB Pi. It utilizes **YOLO26 Nano**, which employs an NMS-free (Non-Maximum Suppression) architecture, bypassing heavy CPU post-processing for bounding box generation.

To prevent the Pi from reaching its 70°C thermal throttling limit during continuous 30 FPS processing, a custom **Unique Frames Processing Algorithm** was developed. It downsamples incoming frames to a 64×64 matrix and computes absolute mean pixel deviation (δ) against a cached baseline. Frames with a deviation below threshold $\tau = 4.5$ are flagged as static duplicates and discarded entirely before inference, drastically reducing CPU load and eliminating frame backlog lag.

5.3 ROS 2 Subsumption Control and Kinematics Engine

Navigation decisions are computed via a Subsumption Control Model (Figure 5), strictly prioritizing safety reflexes over cognitive tracking goals.

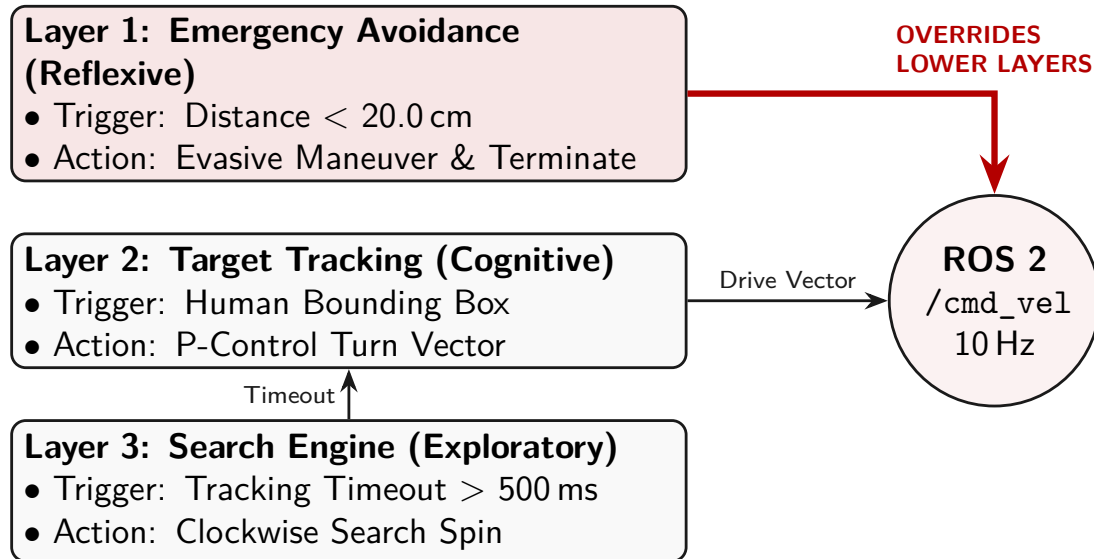


Figure 5: Subsumption Control Model: Layered Priority Tree with Emergency Reflex Override.

The ROS 2 pipeline operates at 10 Hz. If Layer 1 detects an obstacle under 20 cm via the sonars, it commands a reverse vector and truncates execution. Otherwise, Layer 2 engages Proportional Control (P-Control) to center the human target using bounding box coordinates.

The kinematics node relies on the differential drive model to calculate linear and angular velocity vectors from wheel speeds and track width L :

$$v = \frac{v_{\text{right}} + v_{\text{left}}}{2}, \quad \omega = \frac{v_{\text{right}} - v_{\text{left}}}{L}$$

6 Mobile Core Intelligence and Frontend

By repurposing an Android smartphone as the UI frontend, VMO eliminates discrete hardware costs for OLED panels, audio amplifiers, and microphones. The mobile core operates on a dual-pipeline split: it sends low-latency motor commands directly to the Pi via local subnet UDP, while simultaneously syncing logs to a Firebase cloud database.

6.1 Speech Processing and LLM Agent

Speech interaction uses a dual-stage, hardware-safeguarded pipeline (Figure 6). An offline Vosk wake-word engine monitors for the keyword "Veemo". Upon detection, a 400 ms OS debounce releases the microphone lock, handing it to the Google Speech API.

The parsed text is sent to the Gemini 3.1 Flash-Lite API with a strictly constrained system prompt, forcing it to return a JSON object containing three keys: `mood`, `reply`, and `motor`. This guarantees structured agentic behavior, allowing the LLM to control VMO's physical chassis and emotional state seamlessly.

6.2 Locked 60 Hz Vector Rendering

VMO's graphical expressions use a Jetpack Compose Canvas locked at 60 Hz. To maintain a lifelike appearance while stationary, the eye center coordinates are continuously modulated by a low-frequency figure-8 math function ($X_{\text{eye}} = X_{\text{center}} + 12 \cdot \sin(t)$). Emotion transitions utilize native spring-physics animations to prevent brittle linear interpolation.

7 Validation and Conclusion

7.1 Iterative Improvements and Stabilization

During integration, severe electrical noise from the L298N motor high-current switching induced silent packet collisions on the serial data lines. This caused phantom motor commands and corrupted telemetry. The firmware architecture was subsequently retrofitted with **8-bit XOR Checksum Parity** on the PIC-to-ESP32 ASCII link, and **COBS (Consistent Overhead Byte Stuffing)** on the ESP32-to-Pi Protobuf link. These dual layers completely eradicated noise-induced network corruption.

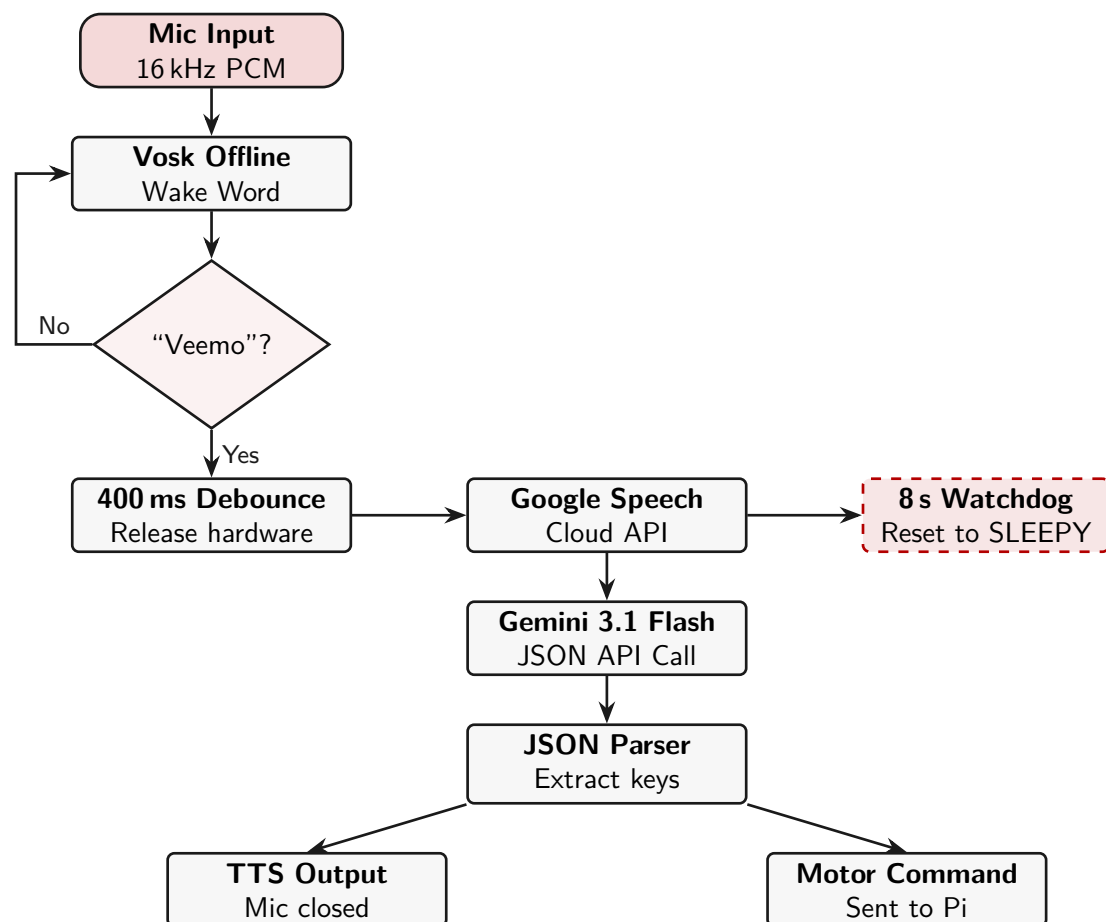


Figure 6: Mobile Speech Processing and LLM Integration Pipeline.

7.2 Conclusion

VMO demonstrates the effective execution of a multi-tier autonomous mobile companion within strict memory, processing, and thermal budgets. By offloading hard real-time tasks to an 8-bit PIC, utilizing a FreeRTOS dual-core ESP32 for queuing, and confining high-level NMS-free YOLO inference to a containerized Pi 4 environment, the platform achieves deterministic reactivity. Substituting discrete displays and voice hardware with an integrated Android edge-node further minimizes build cost while maximizing AI interaction fidelity.

7.3 Future Work

- **Visual-Inertial Odometry (VIO):** Integrating the smartphone's IMU to calculate relative displacement, eliminating wheel slip tracking errors.
- **Automated Vision-Tag Docking:** Using AprilTags for autonomous alignment with a charging station when the battery falls below 15%.
- **On-Device NPU Acceleration:** Porting the YOLO26 Nano graph directly to the smartphone's Neural Processing Unit to bypass the Pi's processing overhead entirely.

A Appendix: Master System Interconnect Diagram

The Master System Interconnect Diagram (SID) below represents the final production-grade hardware blueprint for the VMO platform. It verifies the structural separation of concerns between low-power sensor networks and high-current motor drive axes, highlights the logic level converter lanes bridging the PIC16F877A and ESP32-S3 domains, and maps all onboard telemetry signals.

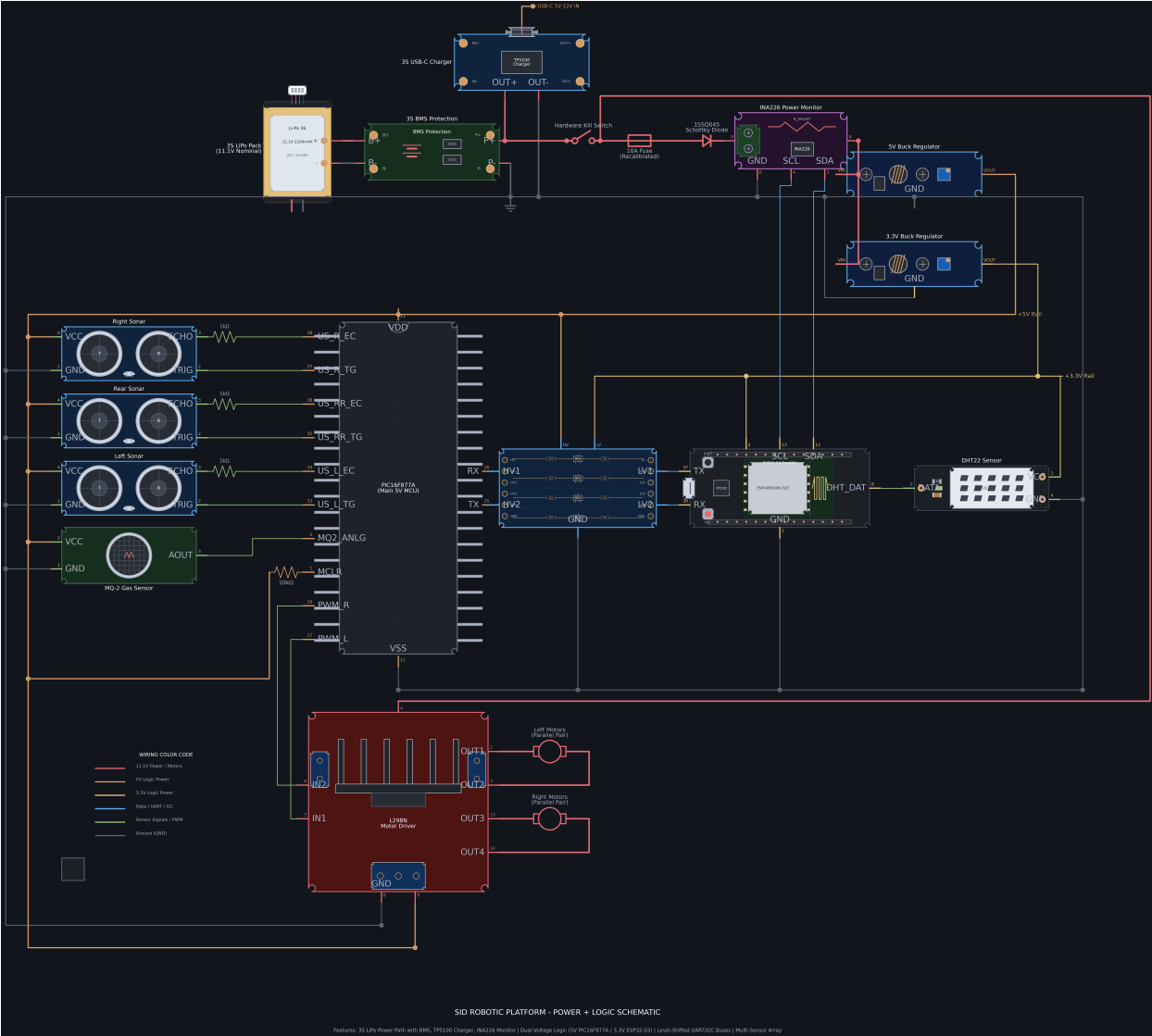


Figure 7: VMO Robotic Platform Master Power and Logic System Interconnect Diagram.